

Task Submission - 31.08.2026

Name: Amma Anjali

Roll No:2303A52385

1. Print the boundary nodes of a binary tree.

The screenshot shows a code editor with a dark theme. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It shows 'Test Cases Passed: 1111 / 1111', 'Attempts: Correct / Total: 1 / 1', 'Accuracy: 100%', 'Points Scored: 4 / 4', 'Time Taken: 0.14', and 'Your Total Score: 182'. On the right, the code for the 'boundaryTraversal' method is visible, implementing a recursive approach to traverse the boundary of a binary tree.

```
12 class Solution {
13     ArrayList<Integer> boundaryTraversal(Node root) {
14         ArrayList<Integer> ans = new ArrayList<>();
15
16         if (root == null)
17             return ans;
18         if (!isLeaf(root))
19             ans.add(root.data);
20         Node curr = root.left;
21         while (curr != null) {
22             if (!isLeaf(curr))
23                 ans.add(curr.data);
24             if (curr.left != null)
25                 curr = curr.left;
26             else
27                 curr = curr.right;
28         }
29         addLeaves(root, ans);
30         ArrayList<Integer> right = new ArrayList<>();
31         curr = root.right;
32         while (curr != null) {
33             if (!isLeaf(curr))
34                 right.add(curr.data);
35             if (curr.right != null)
36                 curr = curr.right;
37             else
38                 curr = curr.left;
39         }
40         for (int i = right.size() - 1; i >= 0; i--)
41             ans.add(right.get(i));
42     }
43 }
44
45 boolean isLeaf(Node node) {
46     return node.left == null && node.right == null;
47 }
48 void addLeaves(Node root, ArrayList<Integer> ans) {
49     if (root == null)
50         return;
51     if (!isLeaf(root))
52         ans.add(root.data);
53     addLeaves(root.left, ans);
54     addLeaves(root.right, ans);
55 }
```

This screenshot is similar to the previous one, showing the same code for the 'boundaryTraversal' method. The 'Output Window' on the left shows the same success message and statistics.

2. Print the left view of a binary tree.

The screenshot shows a code editor with a dark theme. On the left, the 'Output Window' displays 'Problem Solved Successfully' with a green checkmark. It shows 'Test Cases Passed: 1111 / 1111', 'Attempts: Correct / Total: 2 / 2', 'Accuracy: 100%', 'Time Taken: 0.09', and a message: 'You get marks only for the first correct submission if you solve the problem without viewing the full solution.' On the right, the code for the 'leftView' method is visible, implementing a breadth-first search (BFS) approach to print the left view of a binary tree.

```
1 // Structure of Binary Tree Node
2 class Node {
3     int data;
4     Node left, right;
5 }
6 Node(int val) {
7     this.data = val;
8     this.left = this.right = null;
9 }
10
11 class Solution {
12     ArrayList<Integer> leftView(Node root) {
13         ArrayList<Integer> ans = new ArrayList<>();
14         if (root == null)
15             return ans;
16         Queue<Node> q = new LinkedList<>();
17         q.add(root);
18         while (!q.isEmpty()) {
19             int size = q.size();
20             for (int i = 0; i < size; i++) {
21                 Node curr = q.poll();
22                 if (i == 0)
23                     ans.add(curr.data);
24                 if (curr.left != null)
25                     q.add(curr.left);
26                 if (curr.right != null)
27                     q.add(curr.right);
28             }
29         }
30         return ans;
31     }
32 }
33 }
```

3. Print the right view of a binary tree.

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Test Cases Passed

1111 / 1111

Attempts: Correct / Total

1 / 1

Accuracy: 100%

Points Scored

2 / 2

Your Total Score: 184

Time Taken

0.1

Suggest Feedback

```

1
2 Definition for Node
3 class Node {
4     int data;
5     Node left, right;
6
7     Node(int val) {
8         this.data = val;
9         this.left = null;
10        this.right = null;
11    }
12 }
13
14 class Solution {
15     ArrayList<Integer> rightview(Node root) {
16         ArrayList<Integer> ans = new ArrayList<>();
17         if (root == null)
18             return ans;
19         Queue<Node> q = new LinkedList<>();
20         q.add(root);
21         while (!q.isEmpty()) {
22             int size = q.size();
23             for (int i = 0; i < size; i++) {
24                 Node curr = q.poll();
25                 if (i == size - 1)
26                     ans.add(curr.data);
27                 if (curr.left != null)
28                     q.add(curr.left);
29                 if (curr.right != null)
30                     q.add(curr.right);
31             }
32         }
33         return ans;
34     }
35 }

```

Custom Input

Compile & Run

Submit

4. Find the lowest common ancestor (LCA) of two nodes in a binary tree.

Description

Accepted

All Submissions

Runtime

16 ms | Beats 52.53%

Memory

69.70 MB | Beats 88.58%

Code

Java

Editorial

Solutions

Submissions

Code

```

1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode(int x) { val = x; }
8  * }
9  */
10 class Solution {
11     public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
12         if (root == null || root == p || root == q)
13             return root;
14         TreeNode left = lowestCommonAncestor(root.left, p, q);
15         TreeNode right = lowestCommonAncestor(root.right, p, q);
16         if (left == null && right == null)
17             return root;
18         if (left != null) return left;
19         return right;
20     }
21 }

```

Test Result

Testcase

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

5. Find the path from the root to a given node in a binary tree.

Description

Accepted

All Submissions

5 ms | Beats 34.99%

Memory

49.76 MB | Beats 39.07%

Code

Java

Editorial

Solutions

Submissions

Code

```

16 class Solution {
17     public List<String> binaryTreePaths(TreeNode root) {
18         List<String> ans = new ArrayList<>();
19         findPaths(root, "", ans);
20         return ans;
21     }
22     void findPaths(TreeNode root, String path, List<String> ans) {
23         if (root == null)
24             return;
25         path += root.val;
26         if (root.left == null && root.right == null) {
27             ans.add(path);
28             return;
29         }
30         path += "->";
31         findPaths(root.left, path, ans);
32         findPaths(root.right, path, ans);
33     }
34 }

```

Test Result

Testcase

Accepted

Runtime: 3 ms

Case 1

Case 2

Input

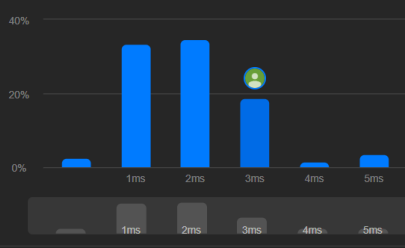
6. Construct a binary tree from inorder and preorder traversal.

Accepted

All Submissions

Runtime
3 ms | Beats 29.51%

Memory
45.28 MB | Beats 93.15%



Code | Java

```
1 class Solution {  
2     public TreeNode buildTree(int[] preorder, int[] inorder) {  
3         return build(preorder, inorder, 0, 0, inorder.length - 1);  
4     }  
5     int preIndex = 0;  
6     TreeNode build(int[] preorder, int[] inorder, int start, int left, int right)  
7     {  
8         if (left > right)  
9             return null;  
10        TreeNode root = new TreeNode(preorder[preIndex++]);  
11        int index = left;  
12        while (inorder[index] != root.val)  
13            index++;  
14        root.left = build(preorder, inorder, start, left, index - 1);  
15        root.right = build(preorder, inorder, start, index + 1, right);  
16        return root;  
17    }  
}
```

Test Result | Testcase

Accepted Runtime: 0 ms

Case 1 Case 2

Input